DSA Pattern
Template Sheet

Java Edition · 10 Patterns · 100 Problems · SDE Interview Prep

KEYWORD → PATTERN MAP

Keyword / Signal	Pattern
"all ways / all paths / generate"	Recursion / Backtracking
"subarray sum / longest subarray"	Prefix Sum + HashMap
"contiguous / window / at most K"	Sliding Window
"sorted / minimize the maximum"	Binary Search on Answer
"next greater / span / histogram"	Monotonic Stack
"tree / path / level / ancestor"	Tree DFS / BFS
"cycle / middle / reverse list"	Linked List Pointers
"top K / Kth / stream / merge K"	Heap / Priority Queue
"schedule / optimal / fractional"	Greedy
"count ways / min cost / can we"	Dynamic Programming

01 ■ Recursion / Backtracking

15 problems

1. Print all subsequences
2. Print all subsets
3. Subset sum problem
4. Combination Sum
5. Combination Sum II
6. Generate Parentheses
7. Permutations of string
8. Permutations of array
9. Unique permutations
10. Letter case permutation
11. N-Queens
12. Rat in a Maze
13. Sudoku Solver
14. Tower of Hanoi
15. Word Search in Grid

WHEN TO USE	Keywords: "all paths", "all ways", "generate all", "every combination",

General Backtracking

```
void solve(state) {
if (baseCondition) { store/return; return; }
for (choice : currentState) {
makeChoice(choice); // choose
solve(newState); // recurse
undoChoice(choice); // BACKTRACK
}
}
```

Subset Pattern

```
void subset(int[] arr, int idx, List path) {
if (idx == arr.length) { print(path); return; }

path.add(arr[idx]); // include
subset(arr, idx+1, path);
path.remove(path.size()-1); // exclude (backtrack)
subset(arr, idx+1, path);
}
```

Permutation Pattern

```
void permute(int[] arr, boolean[] used, List path) {
if (path.size() == arr.length) { print(path); return; }
for (int i = 0; i < arr.length; i++) {
if (!used[i]) {
used[i] = true; path.add(arr[i]);
permute(arr, used, path);
path.remove(path.size()-1); used[i] = false;
}
}
}
```

KEY INSIGHT Every backtracking problem = decision tree. Make a choice, recurse, undo (bac

02 ■ Binary Tree (DFS / BFS)

15 problems

16. Inorder traversal
17. Preorder traversal
18. Postorder traversal
19. Level order traversal
20. Height of binary tree
21. Diameter of binary tree
22. Check balanced tree
23. Lowest Common Ancestor
24. Left view of tree
25. Right view of tree
26. Top view of tree
27. Bottom view of tree
28. Boundary traversal
29. Serialize / Deserialize tree
30. Path sum problems

WHEN TO USE Keywords: "tree", "path", "root to leaf", "level", "ancestor", "view", "hei

DFS — Recursive

```
void dfs(TreeNode node) {
if (node == null) return;

process(node); // preorder
dfs(node.left);
dfs(node.right);
// Move process() for order:
// Before children → Preorder
// Between children → Inorder
// After children → Postorder
}
```

BFS — Level Order

```
Queue q = new LinkedList<>();
q.add(root);
while (!q.isEmpty()) {
int size = q.size(); // level width
for (int i = 0; i < size; i++) {
TreeNode node = q.poll();
process(node);
if (node.left != null) q.add(node.left);
if (node.right != null) q.add(node.right);
}
}
```

KEY INSIGHT DFS for path/depth problems. BFS for level/view problems. Ask: what do I comp

03 ■ Binary Search

10 problems

31. Binary search
32. First & last occurrence
33. Search in rotated array
34. Find peak element
35. Square root of number
36. Aggressive cows
37. Allocate minimum pages
38. Kth smallest element
39. Search in 2D matrix
40. Median of sorted arrays

WHEN TO USE	Keywords: "sorted", "find min/max", "minimize the maximum", "can we

Standard Binary Search

```
int low = 0, high = n - 1;
while (low <= high) {
int mid = low + (high - low) / 2;
if (arr[mid] == target) return mid;
else if (arr[mid] < target) low = mid + 1;
else high = mid - 1;
}
return -1;
```

Search on Answer Space ★

```
int low = minPossible, high = maxPossible, ans = -1;
while (low <= high) {
int mid = low + (high - low) / 2;
if (isValid(mid)) {
ans = mid;
high = mid - 1; // minimize
} else {
low = mid + 1;
}
}
// isValid() → greedy check (cows, pages, capacity)
```

KEY INSIGHT If you can write isValid(mid) and the answer is monotonic (valid above/below a

04 ■ Prefix Sum + Hashing

10 problems

41. Subarray sum = K
42. Longest subarray with sum K
43. Subarray with 0 sum
44. Equilibrium index
45. Count subarrays with given sum
46. Longest consecutive sequence
47. Two sum
48. First non-repeating element
49. Frequency of elements
50. Group anagrams

WHEN TO USE	Keywords: "subarray", "sum equals K", "count subarrays", "longest su

Prefix + HashMap

```
int prefix = 0;
Map<Integer,Integer> mp = new HashMap<>();
mp.put(0, 1); // base: empty prefix

for (int x : array) {
prefix += x;
// if (prefix-k) seen → subarray found
if (mp.containsKey(prefix - k))
answer += mp.get(prefix - k);
mp.put(prefix, mp.getOrDefault(prefix, 0) + 1);
}
// For longest: store first occurrence index instead of count
```

KEY INSIGHT sum(i..j) = prefix[j] - prefix[i-1]. Store prefix sums in a HashMap. Look for prefix-
position.

05 ■ Sliding Window

10 problems

51. Max sum subarray of size K
52. Longest substring no repeats
53. Minimum window substring
54. Count anagrams in string
55. Max consecutive ones
56. Longest subarray with sum <= K
57. K distinct characters substring
58. Sliding window maximum
59. Longest repeating char replacement
60. Subarrays with at most K distinct

WHEN TO USE	Keywords: "contiguous", "substring/subarray", "window of size K", "at

Fixed Window (size = K)

```
int l = 0, sum = 0;
for (int r = 0; r < n; r++) {
sum += arr[r];
if (r - l + 1 == k) {
answer = Math.max(answer, sum);
sum -= arr[l++];
}
}
```

Variable Window (expand / shrink)

```
int l = 0;
for (int r = 0; r < n; r++) {
add(arr[r]); // expand right

while (invalidCondition()) {
remove(arr[l]); // shrink left
l++;
}
updateAnswer(); // window is valid
}
```

KEY INSIGHT Two pointers moving in one direction = O(n). Fixed size: slide mechanically. Va
valid.

06 ■ Stack (Monotonic)

8 problems

61. Valid parentheses
62. Next greater element
63. Next smaller element
64. Stock span problem
65. Largest rectangle histogram
66. Min stack
67. Trapping rainwater
68. Celebrity problem

WHEN TO USE	Keywords: "next greater/smaller", "span", "histogram", "matching brac

Next Greater Element

```
Deque<Integer> stack = new ArrayDeque<>();
int[] ans = new int[n];

for (int i = n - 1; i >= 0; i--) {
while (!stack.isEmpty() && stack.peek() <= arr[i])
stack.pop(); // pop smaller

ans[i] = stack.isEmpty() ? -1 : stack.peek();
stack.push(arr[i]);
}
// For NSE: flip condition to >= | For prev: L→R
```

KEY INSIGHT Monotonic stack maintains decreasing (or increasing) order. When an element
popped elements.

07 ■ Linked List

8 problems

69. Reverse linked list
70. Middle of linked list
71. Detect cycle
72. Merge two sorted lists
73. Remove Nth node from end
74. Palindrome linked list
75. Intersection of linked lists
76. Add two numbers

WHEN TO USE	Keywords: "linked list", "cycle", "middle", "reverse", "merge", "Nth from

Reverse Linked List

```
ListNode prev = null, curr = head;
while (curr != null) {
ListNode nxt = curr.next; // save next
curr.next = prev; // reverse link
prev = curr; // advance prev
curr = nxt; // advance curr
}
return prev; // new head
```

Floyd's Cycle Detection

```
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next;
fast = fast.next.next;
if (slow == fast) return true; // cycle!
}
return false;
// Slow-fast also finds middle:
// when fast reaches end, slow = middle
```

KEY INSIGHT Most LL problems use 2-3 pointer tricks. Always draw pointer movement before edge cases.

08 ■ Greedy

8 problems

77. Activity selection
78. Fractional knapsack
79. Job sequencing
80. Minimum platforms
81. Gas station problem
82. Coin change (greedy)
83. Huffman coding
84. Max meetings in one room

WHEN TO USE	Keywords: "maximum/minimum", "optimal", "select/schedule", "no DP

Greedy Template

```
// Step 1: Sort by optimal criteria
// (end time, ratio, deadline, etc.)
Arrays.sort(items, comparator);

State state = initial;
for (Item item : items) {
if (canTake(item, state)) {
take(item);
update(state);
}
}
// Prove: local optimal choice → global optimal
```

KEY INSIGHT If fractional is allowed → greedy works. If must take whole item → likely 0/1 Kna
time.

09 ■ Heap / Priority Queue

8 problems

85. K largest elements
86. K smallest elements
87. Merge K sorted arrays
88. Sort nearly sorted array
89. Top K frequent elements
90. Median of data stream
91. Connect ropes with min cost
92. Kth largest in stream

WHEN TO USE	Keywords: "top K", "Kth largest/smallest", "stream", "merge K lists", "c

Top K Pattern (min-heap of size K)

```
PriorityQueue<Integer> minH = new PriorityQueue<>();

for (int x : array) {
minH.offer(x);
if (minH.size() > k)
minH.poll(); // remove smallest
}
// heap contains K largest | top = Kth largest
// For K SMALLEST → use max-heap (reverseOrder)
```

Median of Stream (two heaps)

```
PriorityQueue<Integer> maxH = // left half
new PriorityQueue(Collections.reverseOrder());
PriorityQueue<Integer> minH = // right half
new PriorityQueue();

void insert(int x) {
maxH.offer(x); minH.offer(maxH.poll());
if (minH.size() > maxH.size()) maxH.offer(minH.poll());
}
double median() {
return maxH.size() == minH.size()
? (maxH.peek() + minH.peek()) / 2.0 : maxH.peek();
}
```

KEY INSIGHT When you need the Kth element dynamically, a heap of size K beats sorting. Tw

10 ■ Dynamic Programming

12 problems

93. Fibonacci DP
94. Climbing stairs
95. 0/1 Knapsack
96. Subset sum
97. Equal partition subset
98. Coin change
99. Longest increasing subsequence
100. Longest common subsequence
101. Matrix chain multiplication
102. Edit distance
103. Palindrome partitioning
104. Rod cutting problem

WHEN TO USE	Keywords: "count ways", "minimum/maximum cost", "can we achieve substructure

Memoization (Top-Down)

```
int[] dp = new int[n];
Arrays.fill(dp, -1); // -1 = not computed

int solve(int i) {
if (i >= n) return 0; // base case
if (dp[i] != -1) return dp[i]; // cached

int take = solve(i + 1);
int notTake = solve(i + 2);
return dp[i] = Math.max(take, notTake);
}
```

Tabulation (Bottom-Up)

```
dp[0] = baseValue;
for (int i = 1; i <= n; i++)
dp[i] = transition(dp[i-1], dp[i-2]);

// 2D DP (LCS, Knapsack, Edit distance):
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= m; j++) {
dp[i][j] = f(dp[i-1][j],
dp[i][j-1],
dp[i-1][j-1]);
}
}
```

KEY INSIGHT Every DP = recursion + memoization. Start with brute force → add dp[] cache → case, transition.